

DuckDB Extensions

Giving Your Database New Superpowers

Table of Contents

1. What Is an Extension?
2. Built-In vs. Add-On Features
3. Checking What You Have
4. INSTALL and LOAD — The Manual Way
5. Autoloading — The Easy Way
6. `httpfs` — Read Files Straight from the Web
7. `json` — Nested Data, Already Included
8. `excel` — Talk Directly to Spreadsheets
9. A Peek at the Wider Ecosystem
10. Staying Safe
11. Cheat Sheet + Practice Exercise

What Is an Extension?

DuckDB's **core** is small and fast on purpose. Not every feature anyone might want is built in from day one.

Instead, extra features live in **extensions** — optional add-ons you turn on only when you need them.

👉 Same idea as installing an app on your phone: the operating system stays lean, and you add only what you actually use.

Today: reading files from the internet, working with JSON, and reading/writing Excel spreadsheets — all through extensions.

Built-In vs. Add-On Features

Some things work immediately, with **zero setup**:

```
D SELECT * FROM read_csv('data/students.csv'); -- always works
D SELECT * FROM read_json('data/students.json'); -- always works
```

Others need to be **added** before DuckDB can use them:

```
D SELECT * FROM 'https://duckdb.org/data/prices.parquet';
-- needs the httpfs extension
```

`csv`, `parquet`, and `json` ship **bundled** with DuckDB. `httpfs` and `excel` do not — DuckDB fetches them the first time you need them.

Checking What You Have

`duckdb_extensions()` is a table function — query it like any table:

```
D SELECT extension_name, loaded, installed
   FROM duckdb_extensions()
  WHERE extension_name IN ('json','parquet','httpfs','excel','spatial');
```

extension_name	loaded	installed
excel	false	false
httpfs	false	false
json	true	true
parquet	true	true
spatial	false	false

`installed` = downloaded to your machine. `loaded` = active in
this session.

INSTALL and LOAD — The Manual Way

Two separate steps, two separate jobs:

```
D INSTALL https;    -- download it (once – saved to your machine)
D LOAD https;       -- turn it on (every new duckdb session)
```

- `INSTALL` needs internet the **first** time only.
- `LOAD` is cheap and instant, but only lasts for your **current** session — run it again next time you open `duckdb`.

👉 Recall `~/.duckdbrc` from the command-line tutorial? That's exactly where you'd put `LOAD` commands you want to run every time.

Autoloading — The Easy Way

Modern DuckDB (yours included) usually does this **for you**:

```
D SELECT * FROM 'https://duckdb.org/data/prices.parquet' LIMIT 3;
```

The first time this runs, DuckDB notices it needs `httpfs`, silently installs it, loads it, and answers your query — no `INSTALL` / `LOAD` typed by you at all.

⚠ Autoloading still needs internet **the first time** a given extension is used on a machine. After that, it's cached locally.

https — Read Files Straight from the Web

⚠ This slide needs Wi-Fi to run live.

```
D SELECT * FROM 'https://duckdb.org/data/prices.parquet' LIMIT 5;
```

ticker	when	price
APPL	2001-01-01 00:00:00	1
APPL	2001-01-01 00:01:00	2
APPL	2001-01-01 00:02:00	3
MSFT	2001-01-01 00:00:00	1
MSFT	2001-01-01 00:01:00	2

No download step. The file lives on DuckDB's own server; the `https://` URL **is** the table.

https — Same SQL You Already Know

GROUP BY doesn't care whether the file is local or remote:

```
D SELECT ticker, COUNT(*) AS n, MAX(price) AS max_price
   FROM 'https://duckdb.org/data/prices.parquet'
  GROUP BY ticker
 ORDER BY ticker;
```

ticker	n	max_price
APPL	3	3
G00G	3	3
MSFT	3	3

👉 The extension only changes **where DuckDB can look**. Every SQL skill you already have still applies.

json — Nested Data, Already Included

json ships bundled — no `INSTALL`, no `LOAD`, ever. Query `data/students.json` (a plain JSON array of student records) directly, same as any CSV:

```
D SELECT name, gpa
   FROM read_json('data/students.json')
  WHERE gpa > 3.6
 ORDER BY gpa DESC;
```

name	gpa
Hana	3.95
Charlie	3.9
Alice	3.8
Fiona	3.7

excel — Talk Directly to Spreadsheets

Business students live in `.xlsx` files. DuckDB can read **and** write them with the `excel` extension:

```
D INSTALL excel;  
D LOAD excel;  
D CREATE TABLE students AS SELECT * FROM read_csv('data/students.csv');  
D COPY students TO 'data/students.xlsx' WITH (FORMAT xlsx, HEADER true);
```

That's it — `data/students.xlsx` now exists, ready to open in Excel or Google Sheets.

excel — Reading It Back

```
D SELECT * FROM read_xlsx('data/students.xlsx') LIMIT 3;
```

id	name	major	gpa
1.0	Alice	Marketing	3.8
2.0	Bob	Finance	3.5
3.0	Charlie	Accounting	3.9

⚠ Notice `id` came back as `1.0`, not `1`. Excel has no separate "integer" type — every number is a decimal. Small surprise, good to know before you rely on it.

A Peek at the Wider Ecosystem

DuckDB ships more **core extensions** beyond today's three:

Extension	What it adds
<code>spatial</code>	Maps, coordinates, <code>ST_Distance</code> , <code>ST_Point</code>
<code>fts</code>	Full-text search over text columns
<code>icu</code>	International date/time and collation support
<code>autocomplete</code>	The Tab-completion you've been using in the CLI — also an extension!

Beyond that, **community extensions** (`community-extensions.duckdb.org`) are built by outside developers — more specialized, less officially vetted.

Staying Safe

`INSTALL` downloads and runs **code** — treat it like installing software, not like opening a data file.

- Only install extensions from `duckdb.org` or a source you trust.
- On a locked-down or offline machine, autoloading can be turned off:

```
D SET autoload_known_extensions = false;  
D SET autoinstall_known_extensions = false;
```

- Both default to `true` — DuckDB is helpful by default, but you can always take manual control.

Cheat Sheet

Command	What it does
<code>SELECT * FROM duckdb_extensions();</code>	list installed/loaded extensions
<code>INSTALL name;</code>	download an extension (once)
<code>LOAD name;</code>	turn an extension on (per session)
<code>read_csv/json/parquet(...)</code>	bundled — always available
<code>'https://...file.parquet'</code>	needs <code>httpfs</code> (often autoloaded)
<code>read_xlsx(...)</code> / <code>COPY ... (FORMAT xlsx)</code>	needs <code>excel</code>
<code>SET autoloader_known_extensions=false;</code>	disable autoloading

Practice Exercise (1/2)

In your terminal, `cd` into this tutorial's folder, then start `duckdb`:

1. Run `SELECT * FROM duckdb_extensions() WHERE extension_name = 'json';`
— is it loaded?
2. Query `data/students.json` for students majoring in **'Finance'**
3. `INSTALL` and `LOAD` the `excel` extension

Practice Exercise (2/2)

4. Write `data/students.csv` out to a new file, `my_students.xlsx`
5. Read `my_students.xlsx` back with `read_excel(...)` and confirm the row count matches the original CSV
6. If you have Wi-Fi: query `'https://duckdb.org/data/holdings.parquet'` and find which `ticker` has the single largest `shares` value

Extensions = DuckDB, Customized

Small core. Add exactly the features you need, when you need them.

Resources

duckdb.org/docs/extensions/overview — Official extensions guide

community-extensions.duckdb.org — Community-built extensions

OMIS 105 — Introduction to Database Management Systems — Fall 2026